**DBMS LAB PROGRAM**

**Course Code: BCAIAI103**

**Course Title: Database Fundamentals: SQL and NoSQL**

**E-Commerce / Online Shopping Database Management System**

Design and implement an **E-Commerce/Online Shopping Database Management System**.

The system should maintain information about customers, product categories, products, orders, products included in orders, payments, and employees.

Consider the following requirements:

- A customer can place many orders, but each order belongs to one customer.
- A category can contain many products, but each product belongs to one category.
- An order can contain many products, and a product can be included in many orders.
- The quantity of each product included in an order should be maintained.
- Each order can have payment information.
- An employee may manage other employees. An employee can report to another employee, which can be used to demonstrate a self-referencing relationship.

Use the following attributes while designing the database. **Choose appropriate data types and constraints based on the requirements.**

**Customer:**
Customer_ID, Customer_Name, Email, Phone, City

**Category:**
Category_ID, Category_Name

**Product:**
Product_ID, Product_Name, Price, Stock, Category_ID

**Orders:**
Order_ID, Order_Date, Customer_ID, Total_Amount

**Order_Details:**
Order_ID, Product_ID, Quantity

**Payment:**
Payment_ID, Order_ID, Payment_Date, Amount, Payment_Mode, Payment_Status

**Employee:**
Employee_ID, Employee_Name, Designation, Manager_ID

**Perform the following operations.**

**ER Model**

Draw the **ER diagram** for the E-Commerce system showing the entities, attributes, relationships, primary keys, and cardinalities.

**A. Database and Table Creation**

**1.** Create a database named ECommerceDB.

**2.** Create all six required tables with appropriate data types such as INTEGER, VARCHAR, DECIMAL, DATE, etc., for each attribute. Include all the attributes listed above.

**3.** Identify and establish **PRIMARY KEY** constraints for each table based on the logical requirements.

**4.** Establish **FOREIGN KEY** relationships between tables based on the given business scenario. Document which table references which other table.

**5.** Apply **NOT NULL** constraints to all mandatory fields such as Customer_Name, Product_Name, Price, Stock, Order_Date, Quantity, and other fields where values are mandatory.

**6.** Apply **UNIQUE** constraints on Email in the Customer table and Product_Name in the Product table.

**7.** Apply **CHECK** constraints to ensure that Price > 0, Stock >= 0, and Rating is between 1 and 5. Add the Rating column to the Product table.

**8.** Add Registration_Date to the Customer table and Order_Status to the Orders table, and apply DEFAULT constraints so that they default to today's date and 'Pending', respectively

**9.** Add new columns Last_Updated of type DATE to the Product table and Description of type VARCHAR(200) to the Category table using ALTER TABLE.

**10.** Modify the Customer_Name column in the Customer table to support longer names using ALTER TABLE.

**11.** Remove the Description column from the Category table using ALTER TABLE.

**12.** Create a temporary table named Test_Table and remove it using DROP TABLE.

**B. DML Operations**

**13.** Insert at least **5 suitable records into each table**.

**14.** Update the price of a particular product.

**15.** Delete a customer who has not placed any order.

**C. Basic SELECT Queries**

**16.** Display all the records from the Product table.

**17.** Display only the product name and price of all products.

**18.** Display products whose price is greater than ₹1,000 using the WHERE clause.

**19.** Display all products in descending order of their price using ORDER BY.

**20.** Display the different cities of customers without duplicate values using DISTINCT.

**21.** Display the first five products using LIMIT.

**D. Filtering and Expressions**

**22.** Display products whose names start with the letter **'S'** using LIKE.

**23.** Display products belonging to any of the specified categories using IN.

**24.** Display products whose price is between ₹500 and ₹5,000 using BETWEEN.

**25.** Display customers whose phone number is not available using IS NULL.

**26.** Display the product name, price, and calculated stock value using the arithmetic expression: Price × Stock

**E. String and Date Operations**

**27.** Display all customer names in uppercase.

**28.** Display each product name along with the number of characters in the product name.

**29.** Display orders placed after a specified date.

**F. Joins**

**30.** Display each product along with its corresponding category name using an **INNER JOIN**.

**31.** Display the **Customer_Name, Order_ID, and Total_Amount** of customers who have placed orders, including customers who have not placed any order, using a **LEFT JOIN**.

**32.** Display the **Category_Name, Product_Name, and Price** of categories and their products, including categories that do not contain any products, using a **RIGHT JOIN**.

**33.** Display the **Customer_Name, Order_ID, and Order_Date** of customers and their orders, including customers who have not placed orders and orders that do not have a matching customer, using a **FULL OUTER JOIN**.

**34.** Display the **Product_Name, Price, and Category_Name** of products along with their corresponding category names using a **NATURAL JOIN**.

**35.** Display the **Employee_Name, Designation, and Manager_Name** for each employee along with the employee who manages them using a **SELF JOIN**.

**Note for the lab:** If the laboratory uses MySQL, FULL OUTER JOIN is not directly supported. In that case, the equivalent result can be obtained using appropriate LEFT JOIN, RIGHT JOIN, and UNION operations.

**G. Aggregate Functions**

**36.** Find the total number of products using COUNT().

**37.** Find the total value of all products using SUM().

**38.** Find the average price of products using AVG().

**39.** Find the highest product price using MAX().

**40.** Find the lowest product price using MIN().

**H. GROUP BY and HAVING**

**41.** Display the number of products available in each category using GROUP BY.

**42.** Display only those categories that contain more than two products using HAVING.

**I. Subquery**

**43.** Display the products whose price is greater than the average price of all products using a **subquery**.

**J. View**

**44.** Create a view displaying the customer name, order ID, order date, and total amount for each order.

**K. Index**

**45.** Create an index on an appropriate column that is frequently used for searching.

**L. Transactions**

**46.** Perform an update operation on a product inside a transaction and permanently save the change using COMMIT.

**47.** Perform another update operation on a product inside a transaction and cancel the change using ROLLBACK.

**M. Normalization**

Consider the following unnormalized relation containing order information:

Order_ID, Customer_ID, Customer_Name, Customer_Phone, Product1, Product1_Price, Product2, Product2_Price

**48.** Identify the repeating groups in the given relation and convert it into **First Normal Form (1NF)**.

**49.** Convert the resulting relation into **Second Normal Form (2NF)**.

**50.** Convert the resulting relations into **Third Normal Form (3NF)** and identify the primary and foreign keys.